

From LaTeX to Markdown to Quarto

A friendly introduction to version-controlled writing with VS Code, Git, and Quarto

Teaching handout

2026-06-30

1 Learning goals

By the end of this handout, students should be able to explain:

1. Why plain-text writing is useful for serious documents.
2. How LaTeX, Markdown, notebooks, R Markdown, and Quarto are related.
3. Why Quarto is useful for scientific papers, class reports, office documents, and content creation.
4. How a `.qmd` file differs from a simple `.md` file.
5. How VS Code, **Git**, and Quarto work together in a modern writing workflow.

i Note

This handout is intentionally friendly. Many tool names in technical writing sound intimidating: LaTeX, Markdown, Pandoc, Jupyter, RStudio, Quarto, **Git**, YAML. Most of them are just answers to one repeated question: **How can we write once, revise safely, and publish in many formats?**

2 The big idea: source files and rendered files

A modern Quarto workflow separates the **source** from the **output**.

source files		rendering engine		output files
-----		-----		-----
index.qmd	--->	Quarto / Pandoc	--->	HTML
references.bib			---	PDF
figures/			---	Word DOCX
scripts/			---	slides

The source files are plain text. They are easy to read, easy to compare, and easy to track with **Git**.

The output files are what people read or submit: PDF, Word, HTML, slides, books, reports, or websites.

This is the same idea that programmers use:

```
source code ---> build process ---> final product
```

For writing, the source file might be `index.qmd`, and the final product might be `paper.pdf`, `report.docx`, or `slides.html`.

3 A short history of document tools

The tools in this course did not appear randomly. They evolved as people needed better ways to write, compute, collaborate, and publish.

Era	Tool	Main idea	Why it mattered
Late 1970s	TeX	High-quality digital typesetting	Made beautiful mathematical and technical documents possible on computers.
1980s	LaTeX	Structured document preparation on top of TeX	Let authors focus on sections, equations, citations, and structure instead of low-level typesetting.
1990s	LyX	Graphical structured writing for LaTeX	Helped users write LaTeX-like documents without editing all markup by hand.
2004	Markdown	Readable plain-text writing for the web	Made formatted writing look almost like ordinary text.
2011–2014	IPython / Jupyter notebooks	Code, output, equations, and narrative in one document	Made computational exploration and teaching much more interactive.
2012 onward	R Markdown	Markdown plus executable code for reproducible reports	Helped data analysts combine text, code, figures, and results in one source file.
2022 onward	Quarto	Multilingual scientific and technical publishing	Generalized the R Markdown idea across Python, R, Ju-

Era	Tool	Main idea	Why it mattered
			lia, Observable, VS Code, Jupyter, RStudio, HTML, PDF, Word, slides, books, and manuscripts.

3.1 TeX and LaTeX: beautiful scientific documents

TeX was created by Donald Knuth as a high-quality typesetting system, especially strong for mathematical writing. The TeX Users Group describes itself as an organization for users of the TeX typesetting system created by Donald Knuth, founded in 1980. See the TeX Users Group and its notes on what TeX is.

LaTeX was developed by Leslie Lamport in the early 1980s as a higher-level way to write structured documents using TeX. Instead of manually controlling every detail, the author writes commands such as `\section{}`, `\cite{}`, and `\begin{equation}`. The LaTeX Project maintains historical notes and papers about the evolution of LaTeX: <https://www.latex-project.org/news/2019/04/23/history-papers/>.

A very small LaTeX example:

```
\section{Introduction}

Prior work showed an important result \cite{smith2024}.

\begin{equation}
y = \beta_0 + \beta_1 x + \epsilon
\end{equation}
```

The key idea: **LaTeX is excellent when precise typesetting, equations, and publication-quality PDF output are important.**

3.2 LyX: a graphical front end to structured LaTeX writing

LyX describes itself as a document processor that encourages writing based on the structure of a document, using a WYSIWYM approach: “what you see is what you mean,” not simply “what you see is what you get.” See <https://www.lyx.org/>.

LyX is helpful when a person wants many benefits of LaTeX but prefers a graphical writing environment. It is not the same as Quarto. LyX is closer to “a friendlier way to create LaTeX documents.” Quarto is closer to “a plain-text publishing system that can produce many formats and can run code.”

3.3 Markdown: formatting that still looks like normal text

Markdown was introduced by John Gruber in 2004 as a text-to-HTML formatting tool for web writers. The goal was readable plain text that could be converted to valid HTML. See Gruber's original introduction: https://daringfireball.net/2004/03/introducing_markdown and the Markdown project page: <https://daringfireball.net/projects/markdown/>.

A Markdown example:

```
# Introduction

This is a paragraph.

- First point
- Second point

This word is bold and this word is italic.
```

The key idea: **Markdown is readable even before it is rendered.**

That is why Markdown became popular in README files, websites, blogs, GitHub issues, online forums, note-taking apps, and documentation.

3.4 Notebooks: computation plus explanation

Jupyter notebooks combine live runnable code, narrative text, equations, images, **visualizations**, and rich output. JupyterLab's documentation describes notebooks this way: <https://jupyterlab.readthedocs.io/en/latest/user/notebook.html>. Project Jupyter says it was born out of IPython in 2014 as the project evolved to support interactive data science and scientific computing across **programming languages**: <https://jupyter.org/about>.

A notebook is excellent for exploration:

```
write explanation
run a code cell
look at a plot
change the code
run again
write interpretation
```

The key idea: **notebooks are good for interactive thinking and computational teaching.**

However, notebooks are not always ideal for long manuscripts because notebook files are usually JSON, which can be harder to review with **Git** than plain text. **Quarto** can use notebooks, but it also lets us write notebook-like documents as plain-text `.qmd` files.

3.5 R Markdown: reproducible reports

R Markdown combined Markdown text with executable code chunks. The R Markdown website summarizes the goal as: “Analyze. Share. Reproduce.” It emphasizes reports, presentations, dashboards, and reproducible documents: <https://rmarkdown.rstudio.com/>.

The book *R Markdown: The Definitive Guide* notes that R Markdown was first introduced in the `knitr` package in early 2012. See <https://pkg.yihui.org/rmarkdown-book/>.

A simplified R Markdown example:

The key idea: **the text, the code, and the output can live together.**

This reduces copy-and-paste **errors**. If the data change, we can re-render the report and update the numbers, tables, and figures.

3.6 RStudio and Posit

RStudio became a very popular environment for R programming, data analysis, and R Markdown authoring. The company later changed its name to **Posit**, reflecting a broader focus beyond only R. **Posit** announced the name change during `rstudio::conf(2022)` and described the change on its website: <https://posit.co/blog/rstudio-is-now-posit/>.

This matters historically because Quarto came from the same ecosystem that made R Markdown popular, but Quarto was designed to be broader: Python, R, Julia, Observable, VS Code, Jupyter, RStudio, and more.

3.7 Quarto: the general-purpose publishing layer

Posit announced Quarto in 2022 as a new open-source scientific and technical publishing system and described it as the next generation of R Markdown, rebuilt to support more languages and environments. See <https://posit.co/blog/announcing-quarto-a-new-scientific-and-technical-publishing-system>.

The Quarto website describes Quarto as an open-source scientific and technical publishing system for articles, websites, blogs, books, slides, dashboards, and more, with support for Python, R, Julia, and Observable. See <https://quarto.org/>.

The key idea: **Quarto lets us write in plain text, include computations when needed, and publish to many formats.**

4 Tool family tree

This is not a complete technical genealogy, but it is a useful mental model.

```
TeX
├── LaTeX
│   └── LyX gives a graphical structured-writing interface to LaTeX
Plain text writing
```

```
└─ Markdown
   └─ Pandoc Markdown
      │   └─ Quarto uses Pandoc Markdown as its authoring foundation
   └─ R Markdown
      │   └─ Quarto generalizes many R Markdown ideas beyond only R

Interactive computing
└─ IPython
   └─ Jupyter
      │   └─ .ipynb notebooks
      └─ Quarto can work with notebooks or plain-text .qmd files
```

5 What is a .qmd file?

A .qmd file is a **Quarto Markdown** file.

The simplest explanation:

```
.md = Markdown text
.qmd = Markdown text + metadata + citations + cross-references + optional
      executable code + multi-format rendering
```

Quarto's Markdown basics documentation states that Quarto uses Pandoc Markdown and supports common Markdown syntax. See <https://quarto.org/docs/authoring/markdown-basics.html>.

A minimal .qmd file:

```
---
title: "My First Quarto Document"
author: "Student Name"
format: html
---

# Introduction

This is a paragraph written in Markdown.

This word is bold and this word is italic.

## A list

- Write
- Render
- Revise
```

The part between --- lines is called the **YAML header**. It tells Quarto about the document title, author, output format, bibliography, and other options.

6 .md versus .qmd

Feature	Simple .md	Quarto .qmd
Plain-text writing	Yes	Yes
Headings, lists, links, images	Yes	Yes
YAML metadata	Sometimes	Yes, commonly
Citations from .bib files	Not standard	Yes
Figure and table numbering	Not standard	Yes
Cross-references	Not standard	Yes
LaTeX-style math	Depends on renderer	Yes
Executable code chunks	No	Yes
Render to HTML	Yes, with a tool	Yes
Render to PDF	Needs extra tooling	Yes
Render to Word	Needs extra tooling	Yes
Render to slides/books/websites	Needs extra tooling	Yes

A simple rule:



Tip

Use .md for lightweight notes and README files. Use .qmd when the document needs structure, references, figures, code, multiple output formats, or reproducibility.

7 Basic Quarto syntax

7.1 Headings

```
# Level 1 heading

## Level 2 heading

### Level 3 heading
```

7.2 Emphasis

```
This is **bold**.  
This is *italic*.  
This is `inline code`.
```

7.3 Lists

- Item one
- Item two
- Item three

1. First
2. Second
3. Third

7.4 Links and images

```
[Quarto website](https://quarto.org/)  
  
![A figure caption](figures/my-figure.png)
```

7.5 Equations

Inline math:

```
The model includes the parameter  $\beta_1$ .
```

Display math:

```
$$  
y = \beta_0 + \beta_1 x + \epsilon  
$$
```

Labeled equation:

```
$$  
y = \beta_0 + \beta_1 x + \epsilon  
$$ {#eq-model}  
  
See @eq-model.
```


7.6 Figures and cross-references

Quarto supports **cross-references** for figures, tables, equations, sections, code listings, theorems, and more. See <https://quarto.org/docs/authoring/cross-references.html>.

```
![Relationship between input and output.](figures/result.png){#fig-result}
```

As shown in @fig-result, the output increases with the input.

The label #fig-result gives the figure a name. The reference @fig-result points to it.

7.7 Tables

```
| Group | N | Mean |  
| --- | --- | --- |  
| Control | 50 | 1.2 |  
| Treatment | 52 | 1.8 |
```

: Summary statistics. {#tbl-summary}

See @tbl-summary.

7.8 Citations

A Quarto document can use a BibTeX file.

In the YAML header:

```
bibliography: references.bib
```

In the text:

Prior work showed similar results [@smith2024].

Several studies support this view [@smith2024; @lee2021; @chen2020].

Quarto's citation documentation explains how bibliographies and citation syntax work: <https://quarto.org/docs/authoring/citations.html>.

7.9 Executable code chunks

A regular Markdown code block displays code:

```
```python  
print("This is displayed, not executed.")
```
```

A Quarto code chunk can be executed during rendering:

The `#|` lines are Quarto options. They can control labels, captions, warnings, messages, figure size, and more.

8 Why this matters beyond scientific papers

Quarto is useful for scientific manuscripts, but the same ideas are useful in many normal work situations.

| Person | Example use |
|-----------------------|--|
| Undergraduate student | Lab report, senior thesis, portfolio, class presentation |
| Researcher | Manuscript, reproducible analysis, supplement, preprint, conference slides |
| Office worker | Monthly report, policy memo, dashboard explanation, training material |
| Content creator | Long-form article, tutorial, website, course notes, slide deck |
| Data analyst | Report where figures and tables update automatically |
| Team lead | Version-controlled documentation and decision records |

The common problem is the same:

```
I want a document that is easy to revise,  
easy to review,  
easy to reproduce,  
and easy to publish in more than one format.
```

9 Why plain text is powerful

Plain text is not old-fashioned. It is powerful because computers, humans, and **AI** tools can all work with it.

Plain text is:

- easy to search;
- easy to copy;
- easy to compare with **Git**;
- easy for **AI** tools to edit;
- independent of one company's file format;
- small and portable;

- readable even when the original software is not available.

Binary files such as `.docx` and `.pptx` are useful output formats, but they are not ideal as the only source of truth for a long, evolving project.

! Important

A recommended workflow is: **write and version-control the source, then render the final format.**

10 Where Git fits

Git tracks changes over time. It answers questions such as:

- What changed?
- Who changed it?
- When did it change?
- Can we go back to an earlier version?
- Can we review a proposed change before accepting it?

A Quarto project works well with **Git** because `.qmd`, `.yaml`, `.bib`, `.py`, `.R`, and `.txt` files are plain text.

A simple workflow:

```
git init
git add .
git commit -m "Start Quarto project"

# edit index.qmd

git diff
quarto render
git add index.qmd
git commit -m "Revise introduction"
```

11 Where VS Code fits

VS Code is a text editor and development environment. With the Quarto extension, it becomes a practical environment for writing `.qmd` files.

The Quarto VS Code extension provides syntax highlighting, YAML support, embedded language support, render and preview tools, and live previews for math and diagrams. See <https://quarto.org/docs/tools/vscode/>.

Basic setup:

1. Install Quarto CLI.

2. Install VS Code.
3. Install the Quarto extension for VS Code.
4. Install Python, R, or Julia only if you need code chunks in those languages.
5. Install TinyTeX or another TeX distribution only if you need PDF output.

You usually do **not** need `npm install` for normal Quarto manuscript or report writing.

12 A typical Quarto project

```
my-quarto-project/  
  _quarto.yml  
  index.qmd  
  references.bib  
  figures/  
  data/  
  scripts/  
  notebooks/  
  README.md  
  .gitignore
```

12.1 `_quarto.yml`

The `_quarto.yml` file stores project-level settings.

```
project:  
  type: default  
  output-dir: _output  
  
format:  
  html:  
    toc: true  
  pdf: default  
  docx: default  
  
execute:  
  freeze: auto  
  echo: false  
  warning: false  
  message: false
```

12.2 `index.qmd`

The `index.qmd` file is the main document.

```

---
title: "Project Report"
author: "Student Name"
bibliography: references.bib
---

# Introduction

This is the introduction.

# Methods

This is the methods section.

# Results

See @fig-main.

{#fig-main}

# Discussion

This is the discussion.

```

13 Rendering

Render the document from a terminal:

```
quarto render index.qmd
```

Render to a specific format:

```

quarto render index.qmd --to html
quarto render index.qmd --to pdf
quarto render index.qmd --to docx

```

Preview while editing:

```
quarto preview index.qmd
```

In VS Code, the Quarto extension can run preview and render commands from the editor. The VS Code documentation says the extension includes integrated render and preview for Quarto documents: <https://quarto.org/docs/tools/vscode/>.

14 Manuscripts in Quarto

Quarto also has a `manuscript` project type for scholarly articles. The Quarto manuscript documentation says manuscripts can use one or more notebooks or `.qmd` documents as the source of content and computations, and can publish those computations alongside the manuscript. See <https://quarto.org/docs/manuscripts/>.

A manuscript project usually contains:

```
paper/  
  _quarto.yml  
  index.qmd  
  references.bib  
  figures/  
  data/  
  scripts/
```

A simple manuscript configuration:

```
project:  
  type: manuscript  
  
manuscript:  
  article: index.qmd  
  
format:  
  html: default  
  pdf: default  
  docx: default
```

15 A comparison for students

| Tool | Best mental model | Strength | Limitation |
|--------------------|-----------------------------------|--|--------------------------------------|
| Word / Google Docs | Visual document editor | Easy for everyday writing and comments | Harder to version-control precisely |
| LaTeX | Professional typesetting language | Excellent math and PDF quality | Steeper learning curve |
| LyX | Graphical structured LaTeX editor | Easier LaTeX-like writing | Less plain-text/Git-centered |
| Markdown | Simple readable text formatting | Easy to learn, great for notes | Limited scholarly features by itself |

| Tool | Best mental model | Strength | Limitation |
|------------------|------------------------------------|--|--|
| Jupyter Notebook | Interactive computational notebook | Great for exploration and teaching | JSON file can be harder to diff/review |
| R Markdown | Reproducible R-centered reports | Strong data-reporting workflow | Historically centered on R/RStudio |
| Quarto | Plain-text publishing system | Multi-language, multi-format, Git-friendly | Still requires learning source-based writing |
| Git | Version history system | Tracks and reviews changes | New users need practice |
| VS Code | Editor/workbench | Good extensions and Git integration | Can look busy at first |

16 An analogy

Imagine writing a recipe.

A Word file is like writing the final cookbook page directly.

A LaTeX file is like giving a professional printer precise instructions.

A Markdown file is like writing the recipe in clean plain text.

A notebook is like cooking while recording notes, measurements, photos, and experiments.

A Quarto file is like a recipe that can become a web page, printable PDF, Word handout, slide deck, or reproducible lab report.

Git is the notebook that remembers every draft.

VS Code is the desk where all these files are edited.

17 Why Quarto is a good teaching tool

Quarto introduces students to several important modern skills without requiring them to become professional software engineers.

Students practice:

- writing clearly in plain text;
- separating content from appearance;
- using citations and references;
- using figures, tables, and equations;
- rendering the same source to different formats;
- tracking revisions with **Git**;
- making reproducible reports;
- working with **AI** tools more safely because changes can be reviewed.



Tip

A useful message for students: **You do not need to memorize every tool name. Learn the workflow: write source, render output, track changes.**

18 Mini-demonstration for class

18.1 Step 1: Create a file

Create a file named `demo.qmd`.

```
---
title: "My First Quarto Demo"
author: "Student Name"
format:
  html: default
  docx: default
---

# Introduction

This is a Quarto document.

It is written in plain text, but it can become HTML, Word, or PDF.

# A figure

![[A sample figure.]](figures/sample.png){#fig-sample}

See @fig-sample.

# An equation

$$
y = mx + b
$$ {#eq-line}

See @eq-line.
```

18.2 Step 2: Render it

```
quarto render demo.qmd --to html
quarto render demo.qmd --to docx
```


18.3 Step 3: Track the change

```
git init
git add demo.qmd
git commit -m "Create first Quarto demo"
```

18.4 Step 4: Make an edit

Change the introduction, then run:

```
git diff
```

Students should see that **Git** can show the exact sentence-level changes because the source is plain text.

19 Suggested classroom exercise

Ask students to create a short report with:

1. a title and author;
2. two sections;
3. one equation;
4. one image;
5. one table;
6. one cross-reference;
7. one **Git** commit before revision and one Git commit after revision;
8. HTML and Word output.

Optional extension:

- Add a bibliography file.
- Add one citation.
- Add a Python or R code chunk that creates a figure.
- Submit the `.qmd` source and the rendered HTML or **DOCX**.

20 Key takeaways

1. LaTeX made high-quality scientific typesetting widely available.
2. Markdown made formatted writing readable as plain text.
3. **Jupyter notebooks** made computation and explanation interactive.
4. R Markdown made reproducible data reports practical.
5. Quarto combines many of these ideas into one modern publishing system.
6. VS Code gives a practical editing environment.
7. **Git** gives document history and review.
8. Plain text makes writing more transparent, portable, and **AI**-friendly.

21 Sources and further reading

- Quarto home page: <https://quarto.org/>
- Quarto Markdown basics: <https://quarto.org/docs/authoring/markdown-basics.html>
- Quarto **cross-references**: <https://quarto.org/docs/authoring/cross-references.html>
- Quarto citations: <https://quarto.org/docs/authoring/citations.html>
- Quarto manuscripts: <https://quarto.org/docs/manuscripts/>
- Quarto with VS Code: <https://quarto.org/docs/tools/vscode/>
- **Posit** announcement of Quarto: <https://posit.co/blog/announcing-quarto-a-new-scientific-and-technical-publishing-system>
- R Markdown: <https://rmarkdown.rstudio.com/>
- *R Markdown: The Definitive Guide*: <https://pkg.yihui.org/rmarkdown-book/>
- Project Jupyter: <https://jupyter.org/>
- Project Jupyter about page: <https://jupyter.org/about>
- JupyterLab notebook documentation: <https://jupyterlab.readthedocs.io/en/latest/user/notebook.html>
- Markdown original project: <https://daringfireball.net/projects/markdown/>
- John Gruber's Markdown announcement: https://daringfireball.net/2004/03/introducing_markdown
- LyX: <https://www.lyx.org/>
- TeX Users Group: <https://tug.org/>
- LaTeX Project history notes: <https://www.latex-project.org/news/2019/04/23/history-papers/>
- **Posit** name change from RStudio: <https://posit.co/blog/rstudio-is-now-posit/>

22 Instructor note

This handout can be used in two ways:

1. As a reading assignment before a hands-on Quarto lab.
2. As a live demonstration file: open it in VS Code, preview it, edit it, render it, and commit changes with **Git**.

The most important concept to reinforce is not the name of any single tool. The most important concept is the workflow:

```
plain-text source
+ reproducible rendering
+ version control
= safer, clearer, more reusable writing
```